Parte B - Segmentación Semántica con U-Net y Transfer Learning

Objetivos

- Entender la diferencia entre clasificación de imágenes y segmentación semántica (etiqueta por píxel).
- Construir los bloques fundamentales de U-Net en PyTorch: DoubleConv, UpBlock y skip connections.
- Comprender por qué las funciones de pérdida estándar fallan ante desbalance de clases y cómo resolverlo.
- Conectar lo aprendido en el workshop anterior: reutilizar un encoder ResNet-18 preentrenado dentro de una U-Net.
- Evaluar modelos de segmentación con métricas apropiadas: IoU y Dice score.

Descripción de los modelos a desarrollar para el laboratorio

	Modelo	Descripción
M1	U-Net	U-Net desde cero (encoder aleatorio, línea base).
M2	ResNetUNet	ResNetUNet con encoder congelado (solo se entrena el decoder).
M3	ResNetUNet	ResNetUNet con fine-tuning completo (encoder + decoder).

Notebook Lab_2_UNet_Workshop.ipynb

Actividad 1 — Dataset y transforms sincronizados

Usar el Oxford-IIIT Pet Dataset (https://www.kaggle.com/datasets/tanlikesmath/the-oxfordiiit-pet-dataset), que incluye imágenes de mascotas con máscaras de segmentación píxel a píxel. Cada píxel tiene una de tres etiquetas: primer plano (el animal), fondo, o borde ambiguo. Lo tratarás como un problema binario: el píxel pertenece al animal o no.

Antes de escribir código, ejecuta show_sample sobre al menos 3 imágenes y observa cómo se ve la máscara cruda y el overlay binario. Entiende qué representa cada valor numérico en la máscara antes de continuar.

El detalle crítico de esta actividad es que imagen y máscara deben recibir exactamente las mismas transformaciones geométricas. Si aplicas un flip o una rotación de forma independiente, la máscara

queda desalineada con la imagen. La clase PetSegDataset ya implementa esto: estudia cómo lo hace antes de usarla.

Pregunta: ¿Por qué se usa interpolación NEAREST para la máscara y BILINEAR para la imagen al hacer resize? ¿Qué pasaría si usaras bilineal para ambas?

Actividad 2 — Construir U-Net desde cero

La U-Net tiene forma de «U»: un encoder que reduce la resolución espacial extrayendo features, y un decoder que la recupera progresivamente. La clave son las skip connections, que conectan cada nivel del encoder con su nivel correspondiente en el decoder mediante concatenación.

Los bloques DoubleConv y UpBlock ya están implementados. Tu tarea es completar el método forward de la clase UNet. El encoder ya está escrito y guarda las skip connections en una lista llamada skips. Debes iterar sobre self.decoders usando esas skips en orden inverso: la última skip del encoder corresponde al primer bloque del decoder.

Verifica tu implementación con la prueba de shapes antes de continuar. La salida debe tener shape [2, 1, 256, 256] para una entrada [2, 3, 256, 256].

Pregunta: ¿Qué información aportan las skip connections que el decoder no podría recuperar por sí solo?

Actividad 3 — Funciones de pérdida para segmentación

Con CrossEntropyLoss estándar existe un problema serio en segmentación: si el 90% de los píxeles son fondo, el modelo puede ignorar completamente el objeto de interés y aun así tener 90% de accuracy. Esto es especialmente crítico en imágenes médicas.

Estudia la implementación de DiceLoss y BCEDiceLoss. Entiende por qué el coeficiente Dice es robusto ante el desbalance de clases y cómo la pérdida combinada BCE + Dice estabiliza el entrenamiento. No modifiques estas clases todavía, pero ten claro qué recibe cada una como entrada y qué devuelve, porque las usarás en todas las actividades siguientes.

Pregunta: ¿Por qué CrossEntropyLoss falla ante desbalance de clases? ¿Qué mide exactamente el coeficiente Dice?

Actividad 4 — Entrenar U-Net como línea base

Instancia UNet(in_channels=3, num_classes=1) y entrénala con train_seg por 5 epochs usando la pérdida combinada y Adam con lr=1e-3. Esta es tu línea base: M1 con encoder aleatorio, sin ningún conocimiento previo.

Guarda las curvas de loss, IoU y Dice. No necesitas convergencia completa, solo un punto de referencia numérico para comparar contra los modelos con encoder preentrenado.

Actividad 5 — ResNetUNet: encoder preentrenado como backbone

Aquí está la conexión directa con el workshop anterior. En lugar de aprender el encoder desde cero, reutilizas los pesos de ResNet-18 preentrenado en ImageNet. El encoder ya sabe extraer features útiles; el decoder aprende a usarlas para producir la máscara.

Tu tarea es completar el método forward de ResNetUNet. El encoder ya extrae features en cinco niveles (s0 a s4) con sus respectivas resoluciones y canales. Debes implementar el decoder usando dec4 hasta dec0, pasando cada skip connection en el orden correcto. Verifica shapes antes de entrenar.

Luego congela los parámetros del encoder (enc0, enc1, enc2, enc3, enc4) y verifica con summary cuántos parámetros quedan entrenables. Entrena solo el decoder por 5 epochs con lr=1e-3. Guarda el modelo.

Pregunta: ¿En qué se diferencia congelar el encoder aquí respecto a lo que hiciste con VGG en el workshop anterior?

Actividad 6 — Fine-tuning end-to-end

Carga el modelo guardado de la actividad anterior y descongela el encoder. Entrena 5 epochs adicionales con lr=1e-4, un learning rate más pequeño para no arruinar los pesos preentrenados. Compara las curvas de esta fase con las de la fase congelada en TensorBoard.

Actividad 7 — Evaluación comparativa y visualización

Con los tres modelos entrenados, evalúa IoU y Dice sobre el test set completo y reporta los resultados en una tabla. Luego usa visualize_predictions para inspeccionar visualmente las predicciones de cada modelo: observa especialmente qué tan precisos son los bordes y dónde aparecen falsos positivos o falsos negativos.

Pregunta: ¿Dónde falla más cada modelo: en el interior de la región o en los bordes? ¿A qué atribuyes esa diferencia?

Preguntas de investigación: Responde en celdas Markdown apoyándote en la documentación, papers o experimentos propios.

1. Entrena usando solo BCEWithLogitsLoss sin Dice. ¿Qué pasa con las métricas? ¿Qué estrategias existen para manejar el desbalance en segmentación médica?
2. Elimina las skip connections de ResNetUNet y entrena. ¿Qué observas visualmente en los bordes de las predicciones?
3. La función compute_metrics usa threshold=0.5 para binarizar. Grafica la curva IoU vs. threshold en el test set. ¿Cuál sería el threshold óptimo?
4. Investiga qué augmentations son estándar en imágenes médicas (elastic deformation, gamma correction). Implementa al menos una en PetSegDataset y mide el impacto.
5. Reto opcional: implementa la métrica Hausdorff distance. ¿Qué mide que IoU y Dice no capturan?

Entregables

Archivo — Notebook (grupoX_LAB#2B.ipynb)

Un único notebook .ipynb con todo el código completo y ejecutado, que incluya las respuestas a todas las preguntas en **celdas Markdown.**

El reporte debe ser autocontenido: alguien que no ejecute el notebook debe poder entender la solución planteada.